Jaden Leonard

Dr. Sarker

COSC 514 – Operating Systems

10 May 2026

Individual Reflection: Subsystem A – Process Management & CPU Scheduling

Building Subsystem A pushed me to think carefully about data structure design, algorithm selection, and API stability. Here's what I decided, why, and where the current implementation falls short.

I represented each process as a single PCB struct holding both static attributes (arrival time, burst time, priority) and runtime state (remaining time, waiting time, turnaround time). Every scheduling algorithm needs all of this, so keeping it in one place avoided unnecessary indirection. Splitting static and dynamic data into separate structs would have added complexity with no real benefit at this scale.

Priority scheduling is non-preemptive by design. A preemptive version would require interrupting a running process mid-burst, which in a real OS means saving register state. In a logical simulator there is no execution context to save, so the added complexity would not model anything more meaningful than the non-preemptive version already does.

One unexpected issue was a naming conflict between my SCHED_RR enum value and a POSIX macro of the same name defined in <pthread.h>. I resolved it by prefixing all algorithm enum values with ALGO_, which also reads more clearly and avoids future collisions with system-level identifiers.

The spec required that API functions never print directly. This shaped the GanttEntry struct and the two-mode design of sched_get_gantt(), where the first call returns the entry count

and the second fills a caller-allocated buffer. This keeps the subsystem decoupled from any output format, which matters in Part II when a unified CLI handles all printing.

The main limitation is that time is modeled as discrete integers, so burst and arrival times must be whole numbers. The Priority scheduler also has no aging, meaning low-priority processes can starve if higher-priority ones keep arriving. Given more time, adding aging and implementing MLFQ would be the natural next steps.

The 73-test suite covers all functions, edge cases, and error paths, which should make integration with the memory and synchronization subsystems in Part II straightforward.